

INFORME DE AUDITORÍA DE SOFTWARE — Estok Inventory System

****Fecha:**** 15/07/2026

****Auditor:**** Cline (Auditor de Software Experto)

****Versión del código:**** commit `2e4fcc10f8d266f0fccf7bef87bccd8bac6d24e1`

1. ESTADO DE LAS FUNCIONALIDADES

1.1 Funcionalidades terminadas y 100% operativas

Funcionalidad	Archivos que la componen	Estado
--- --- ---		
Autenticación JWT (login/register/refresh)	`config/urls.py` (TokenObtainPairView, TokenRefreshView, TokenVerifyView), `frontend/src/services/auth.ts`	☒ Completo
CRUD de Estoks (multi-tenant)	`inventario/api/viewsets/estok.py` (EstokViewSet), `inventario/models.py` (Estok, Membresia), `frontend/src/services/auth.ts` (crearEstok, unirseConCodigo)	☒ Completo
CRUD de Ubicaciones	`inventario/api/viewsets/organizacion.py` (UbicacionViewSet), `inventario/api/serializers/organizacion.py`	☒ Completo
CRUD de Contenedores + QR	`inventario/api/viewsets/organizacion.py` (ContenedorViewSet), `inventario/services/qr_service.py`, `inventario/models.py` (Contenedor.save genera QR automático)	☒ Completo
CRUD de Objetos con herencia multi-tabla	`inventario/api/viewsets/objetos.py` (ObjetoViewSet), `inventario/api/serializers/objetos.py` (ObjetoCreateSerializer con multi-tabla), `inventario/models.py` (Objeto + LibroRevista, Tecnologia, MuebleArte, Ropa)	☒ Completo
Filtros de objetos (tipo, ubicación, estado, búsqueda)	`inventario/api/viewsets/objetos.py` (get_queryset con 10+ filtros)	☒ Completo
Subida de fotos a objetos	`inventario/api/viewsets/objetos.py` (subir_foto action), `inventario/api/serializers/multimedia.py` (FotoObjetoUploadSerializer)	☒ Completo
Historial de precios + plusvalía	`inventario/services/stock_service.py` (StockValuationService), `inventario/api/viewsets/objetos.py` (actualizar_precio, historial_precios, plusvalia)	☒ Completo
Alertas de stock	`inventario/services/stock_service.py` (crear_alerta_stock, actualizar_cantidad_stock), `inventario/api/viewsets/objetos.py` (crear_alerta_stock)	☒ Completo
Owner Action (vender/conservar/tirar)	`inventario/api/viewsets/objetos.py` (owner_action, clear_owner_action)	☒ Completo
Exportación CSV	`inventario/api/viewsets/objetos.py` (exportar_csv)	☒ Completo
Estadísticas del dashboard	`inventario/api/viewsets/objetos.py` (estadisticas)	☒ Completo
Códigos de invitación + unirse a Estok	`inventario/api/viewsets/estok.py` (CodigoInvitacionViewSet, EstokViewSet.unirse), `inventario/models.py` (CodigoInvitacion)	☒ Completo
Chat interno entre miembros	`inventario/api/viewsets/chat.py` (MensajeViewSet), `inventario/models.py` (Mensaje), `frontend/src/services/chat.ts`, `frontend/src/components/ChatModal.astro`	☒ Completo

| **Usuarios online (presencia en tiempo real)** | `inventario/api/viewsets/usuarios.py` (UserViewSet.ping, online, access_log), `frontend/src/services/auth.ts` (ping, fetchOnlineUsers) | ☒ Completo |

| **RBAC dinámico por Estok** | `inventario/api/viewsets/base.py` (HasRolePermission, EsAdminDelEstok), `inventario/models.py` (Role, Membresia) | ☒ Completo |

| **Endpoint de versión** | `inventario/api/viewsets/version.py` (VersionViewSet) | ☒ Completo |

| **Frontend Astro con layout base** | `frontend/src/layouts/BaseLayout.astro`, `frontend/src/layouts/AuthLayout.astro` | ☒ Completo |

| **Componentes UI reutilizables** | `frontend/src/components/LoadingState.astro`, `ErrorState.astro`, `EmptyState.astro`, `BackButton.astro`, `PageHeader.astro`, `ConfirmDialog.astro`, `AuthGuard.astro` | ☒ Completo |

| **Cliente HTTP genérico (api.ts)** | `frontend/src/lib/api.ts` (apiGet, apiPost, apiPut, apiPatch, apiDelete, fetchAllPages) | ☒ Completo |

| **Tipos TypeScript completos** | `frontend/src/types/index.ts` (343 líneas con todos los tipos) | ☒ Completo |

| **Servicio de compresión de imágenes frontend** | `frontend/src/services/image.ts` (comprimirImagenBase64, fileToDataURL, base64ToBlob) | ☒ Completo |

| **Migración 0009 (eliminar MercadoLibreToken)** | `inventario/migrations/0009\_remove\_mercadolibretoken.py` | ☒ Completa |

1.2 Funcionalidades implementadas pero que NO funcionan o están incompletas

| Funcionalidad | Problema | Archivos afectados |

---|---|---

| **Análisis con IA local (LM Studio)** | Depende de LM Studio corriendo en `localhost:1234` con el modelo `qwen2.5-vl-7b-instruct`. En producción (Coolify/Hetzner) NO hay GPU AMD Radeon, por lo que LM Studio **no puede correr**. El endpoint `analizar\_con\_ia` y `analizar\_imagen` con motor='local' fallarán siempre en producción. | `inventario/services/ai\_vision\_service.py` (LMStudioClient), `inventario/api/viewsets/objetos.py` (analizar_con_ia, analizar_imagen) |

| **Análisis con Gemini** | Depende de la variable de entorno `GEMINI\_API\_KEY`. Si no está configurada en el servidor, el motor 'gemini' también falla. **NO VERIFICADO** si está configurada en producción. | `inventario/services/ai\_vision\_service.py` (GeminiClient) |

| **MarketingService (generar anuncios)** | `generar\_anuncios` y `publicar\_en` existen como endpoints, pero **no hay integración real** con APIs de Facebook/Instagram/MercadoLibre. `publicar\_en` solo marca el objeto como "publicado" en un campo JSON, no hace POST real a ninguna plataforma. | `inventario/services/marketing\_service.py`, `inventario/api/viewsets/objetos.py` (generar_anuncios, publicar_en, estado_publicacion) |

| **Búsqueda de precio en MercadoLibre** | El endpoint `buscar\_precio\_ml` usa la API pública de MercadoLibre sin autenticación (fallback si no hay token). El endpoint `buscar\_precio\_referencia` hace scraping de HTML de listado.mercadolibre.com.ar, que es **frágil** (cambios en el HTML rompen el scraping). | `inventario/api/viewsets/objetos.py` (buscar_precio_ml, buscar_precio_referencia), `inventario/services/precio\_referencia\_service.py` |

| **Páginas frontend de objetos** | Las páginas `frontend/src/pages/objetos.astro`, `objetos/[id]/index.astro`, `objetos/[id]/editar/index.astro`, `objetos/nuevo.astro` existen pero **no fueron auditadas en profundidad**. Pueden tener bugs de integración. | `frontend/src/pages/objetos/\*` |

****Página de diagnóstico IA****	`frontend/src/pages/diagnostico-ia.astro` existe pero depende de los endpoints de IA que pueden no funcionar en producción.	`frontend/src/pages/diagnostico-ia.astro`
****Página de escaneo QR****	`frontend/src/pages/escanear/index.astro` existe pero ****no fue auditada****.	`frontend/src/pages/escanear/index.astro`
****seed_data / seed_demo_data****	Los management commands `seed\_data.py` y `seed\_demo\_data.py` existen pero ****no fueron auditados**** para verificar que funcionan con el esquema actual.	`inventario/management/commands/seed\_data.py`, `seed\_demo\_data.py`

1.3 Código muerto o huérfano

Archivo	Problema
****`inventario/api/serializers.py`**** (archivo plano, no el directorio)	Existe un `serializers.py` plano en `inventario/api/` además del directorio `inventario/api/serializers/`. Esto es ****código legacy/muerto**** que ya no se importa desde ningún lado (los imports van a `inventario.api.serializers` como paquete).
****`inventario/migrations/0001\_initial.py`****	Migración inicial que puede estar en conflicto con migraciones posteriores. No se puede determinar si está aplicada o si hay migraciones sin aplicar.
****~100 archivos `tmp\_`****	Hay aproximadamente 100 archivos temporales (`bat`, `py`, `php`, `sql`) en la raíz del proyecto. Son scripts de diagnóstico/deploy que ****contaminan el repositorio****. Deberían estar en `.gitignore` o eliminarse.
****`Dockerfile.backend` y `Dockerfile.frontend`****	Según `clinerules` son LEGACY y no se usan en producción. Sin embargo, siguen existiendo en el repo (no listados pero mencionados en las reglas).
****`nginx.conf`**** (vs `nginx-combined.conf`)	Existen dos configuraciones de Nginx. `nginx-combined.conf` es la que se usa en `Dockerfile.combined`. `nginx.conf` puede ser legacy.
****`generate\_icons.py`****	Script para generar íconos. Se usó una vez y ya no es necesario en producción.
****`tmp\_ml\_response.json`****	Archivo JSON de respuesta de prueba de MercadoLibre.

2. EVALUACIÓN DE CLINERULES Y BUENAS PRÁCTICAS

2.1 Cumplimiento de .clinerules

Regla	Cumplimiento	Observación
****No concatenar comandos con `&&`****	☒ Cumplido	Todos los comandos se ejecutan individualmente
****DB_HOST real de Coolify****	☒ Cumplido	`settings.py` usa `cagtcifjoy8ydxugg4bkdl1` como hostname
****No usar DATABASE_URL****	☒ Cumplido	`settings.py` usa variables DB_* individuales
****Red Docker "coolify"*****	☒ Cumplido	`docker-compose.yml` incluye la red externa
****Un solo contenedor (Dockerfile.combined)****	☒ Cumplido	`Dockerfile.combined` es el único usado
****Entrypoint con Gunicorn + Astro + Nginx****	☒ Cumplido	`entrypoint.sh` lanza los 3 procesos
****Dominio con esquema https://****	☒ Cumplido	`CSRF\_TRUSTED\_ORIGINS` incluye `https://eestok.duckdns.org`

| **Auth centralizado en auth.ts** | ☒ Cumplido | ``getAuthHeaders()`` y ``getToken()`` viven solo en ``frontend/src/services/auth.ts`` |

| **URL de producción siempre la misma** | ☒ Cumplido | No hay referencias a localhost para el backend |

| **NO tocar lo que no pidieron** | ☐ Parcial | Hay evidencia de múltiples cambios no solicitados (refactors de serializers, modularización de viewsets) que pueden haber introducido regresiones |

| **Límite de 3 intentos** | ☐ Parcial | La gran cantidad de archivos ``tmp_*` (especialmente ``tmp_do_deploy*.bat`` y ``tmp_create_token*.bat``) sugiere que se excedieron los 3 intentos en múltiples ocasiones para resolver problemas de deploy |

| **Redeploy lo hace el usuario** | ☒ Cumplido | No hay llamadas automáticas a Coolify API |

| **Reportar honestamente qué se verificó** | ☐ Parcial | Existen archivos ``DIAGNOSTICO_VERIFICADO.md`` y ``DIAGNOSTICO_ANALIZAR_TRABADO.md`` que indican diagnóstico, pero no hay evidencia de verificación en producción de las funcionalidades de IA |

| **Mantener archivos <400-500 líneas** | ☒ Incumplido | ``inventario/api/viewsets/objetos.py`` tiene **930 líneas**, ``inventario/services/ai_vision_service.py`` tiene **905 líneas**, ``inventario/models.py`` tiene **861 líneas**, ``frontend/src/services/auth.ts`` tiene **634 líneas** |

2.2 Consistencia del desarrollo

Aciertos:

- La modularización de viewsets y serializers en paquetes fue un gran acierto arquitectónico
- El sistema de permisos RBAC vía Membresía es sólido y consistente
- La centralización de auth en ``auth.ts`` se respeta en todos los servicios frontend
- Los tipos TypeScript están completos y bien documentados
- El cliente HTTP genérico ``api.ts`` evita duplicación de lógica de fetch

Desviaciones detectadas:

- Exceso de archivos temporales:** ~100 archivos ``tmp_*` en la raíz es una señal de que el proceso de deploy no está estabilizado y se hicieron muchos intentos sin limpiar
- Archivos demasiado grandes:** ``objetos.py`` (930 líneas) y ``ai_vision_service.py`` (905 líneas) violan el límite de 400-500 líneas establecido en ``.clinerules``
- Código muerto no eliminado:** ``serializers.py`` legacy, ``nginx.conf`` legacy, ``Dockerfile.backend`/`Dockerfile.frontend`` legacy
- Scraping frágil:** La dependencia de scraping de HTML de MercadoLibre es inherentemente frágil y debería reemplazarse por API oficial

3. ARQUITECTURA Y MODULARIDAD

3.1 Análisis de acoplamiento

Backend (Django):

Componente	Acoplamiento	Evaluación
---	---	---

| `inventario/api/viewsets/objetos.py` (930 líneas) | ● ****ALTO**** | Este archivo concentra demasiadas responsabilidades: CRUD de objetos, análisis IA, marketing, stock, precios, exportación, estadísticas, fotos. Debería dividirse en múltiples ViewSets o mixins. |

| `inventario/services/ai\_vision\_service.py` (905 líneas) | ● ****ALTO**** | Contiene LMStudioClient, GeminiClient y AIVisionService en un solo archivo. Deberían ser módulos separados. |

| `inventario/models.py` (861 líneas) | ● ****MEDIO**** | Todos los modelos en un solo archivo. Aunque Django lo permite, para un proyecto de este tamaño sería mejor separar en `models/` como paquete. |

| `inventario/api/viewsets/base.py` (102 líneas) | ● ****BAJO**** | Bien modularizado, permisos reutilizables. |

| `inventario/api/serializers/` (paquete) | ● ****BAJO**** | Bien modularizado por dominio. |

| `inventario/services/stock\_service.py` (380 líneas) | ● ****BAJO**** | Bien encapsulado, responsabilidad única. |

| `inventario/services/qr\_service.py` (222 líneas) | ● ****BAJO**** | Bien encapsulado. |

| `inventario/services/marketing\_service.py` (426 líneas) | ● ****BAJO**** | Bien encapsulado, aunque la funcionalidad es limitada (no hay integración real con APIs). |

****Frontend (Astro/TypeScript):****

Componente	Acoplamiento	Evaluación

| `frontend/src/services/auth.ts` (634 líneas) | ● ****MEDIO**** | Demasiadas responsabilidades: auth, tokens, Estoks, roles, códigos de invitación, heartbeat, versión. Debería dividirse. |

| `frontend/src/lib/api.ts` (149 líneas) | ● ****BAJO**** | Bien encapsulado. |

| `frontend/src/types/index.ts` (343 líneas) | ● ****BAJO**** | Bien organizado. |

| `frontend/src/services/chat.ts` (101 líneas) | ● ****BAJO**** | Bien encapsulado. |

| `frontend/src/services/image.ts` (92 líneas) | ● ****BAJO**** | Bien encapsulado. |

3.2 Sugerencias de refactorización

****Prioridad ALTA:****

- **Dividir `objetos.py` (930 líneas)** en:**
 - `ObjetoViewSet` base (CRUD + filtros)
 - `ObjetoIAViewSet` (analizar_con_ia, analizar_imagen)
 - `ObjetoMarketingViewSet` (generar_anuncios, publicar_en, estado_publicacion)
 - `ObjetoStockViewSet` (actualizar_precio, historial_precios, plusvalia, crear_alerta_stock)
 - `ObjetoOwnerViewSet` (owner_action, clear_owner_action)
 - `ObjetoExportViewSet` (exportar_csv, estadisticas)
 - `ObjetoFotoViewSet` (subir_foto)
 - `ObjetoPrecioViewSet` (buscar_precio_ml, buscar_precio_referencia)
 - O usar un solo ViewSet con mixins para mantener las rutas unificadas
- **Dividir `ai\_vision\_service.py` (905 líneas)** en:**
 - `services/vision/\_\_init\_\_.py`
 - `services/vision/lm\_studio\_client.py`
 - `services/vision/gemini\_client.py`

- `services/vision/vision\_service.py` (orquestador)
- `services/vision/vision\_result.py` (dataclass)

3. **Dividir `auth.ts` (634 líneas)** en:

- `services/auth/tokens.ts` (getToken, setTokens, refreshToken)
- `services/auth/session.ts` (login, logout, register)
- `services/auth/estoks.ts` (crearEstok, unirseConCodigo, cambiarEstokActivo)
- `services/auth/presence.ts` (ping, fetchOnlineUsers)
- `services/auth/index.ts` (re-export)

Prioridad MEDIA:

4. **Separar `models.py` en paquete `models/`**:

- `models/\_\_init\_\_.py`
- `models/usuarios.py` (CustomUser, Role)
- `models/estok.py` (Estok, Membresia, CodigoInvitacion)
- `models/ubicacion.py` (Ubicacion, Contenedor)
- `models/objeto.py` (Objeto, LibroRevista, Tecnologia, MuebleArte, Ropa)
- `models/multimedia.py` (FotoObjeto)
- `models/historial.py` (HistorialPrecio, AlertaStock)
- `models/chat.py` (Mensaje)

5. **Eliminar código muerto**:

- `inventario/api/serializers.py` (archivo plano)
- `nginx.conf` (si no se usa)
- Todos los archivos `tmp\_\*.py`
- `generate\_icons.py`

6. **Reemplazar scraping de MercadoLibre** por API oficial de MercadoLibre con autenticación OAuth

4. CONCLUSIÓN Y PRÓXIMOS PASOS RECOMENDADOS

Resumen del estado general

El proyecto **Estok** tiene una arquitectura sólida con:

- ☒ Multi-tenancy vía Estok + Membresia
- ☒ RBAC dinámico y granular
- ☒ Herencia multi-tabla para objetos
- ☒ Chat interno con aislamiento por Estok
- ☒ Frontend modular con Astro + TypeScript
- ☒ Cliente HTTP genérico y centralizado
- ☒ Tipos TypeScript completos y consistentes

Sin embargo, arrastra **deuda técnica significativa**:

- ☒ Archivos demasiado grandes (violación de reglas de arquitectura)

- ✗ ~100 archivos temporales contaminando el repositorio
- ✗ Funcionalidades de IA que probablemente no funcionan en producción (LM Studio requiere GPU local)
- ✗ MarketingService sin integración real con marketplaces
- ✗ Scraping frágil de MercadoLibre
- ✗ Código legacy no eliminado

Prioridades inmediatas

1. ** ● CRÍTICO **: Limpiar los ~100 archivos `tmp\_\*` del repositorio. Esto es contaminación visual y riesgo de seguridad (algunos contienen tokens, passwords, IPs).
2. ** ● CRÍTICO **: Decidir el destino de LM Studio. Si en producción no hay GPU, hay que:
 - Hacer que Gemini sea el motor por defecto (y asegurarse que GEMINI_API_KEY esté configurada)
 - O eliminar el código de LM Studio para no generar falsas expectativas
3. ** ● ALTA **: Refactorizar `objetos.py` (930 líneas) y `ai\_vision\_service.py` (905 líneas) para cumplir con el límite de 400-500 líneas.
4. ** ● ALTA **: Refactorizar `auth.ts` (634 líneas) en módulos más pequeños.
5. ** ● ALTA **: Eliminar código muerto: `serializers.py` legacy, `nginx.conf` legacy, `generate\_icons.py`.
6. ** ● MEDIA **: Separar `models.py` en paquete `models/`.
7. ** ● MEDIA **: Verificar que `GEMINI\_API\_KEY` esté configurada en producción y que el motor Gemini funcione correctamente.
8. ** ● MEDIA **: Agregar `.gitignore` para archivos `tmp\_\*` y `\*.bat`, `\*.php` para evitar que vuelvan a aparecer.
9. ** ● BAJA **: Implementar integración real con APIs de marketplaces (Facebook, Instagram, MercadoLibre) o eliminar MarketingService si no se va a usar.
10. ** ● BAJA **: Reemplazar scraping de MercadoLibre por API oficial con OAuth.

Este informe fue generado con base en el análisis estático del código en el commit `2e4fcc10f8d266f0fccf7bef87bccd8bac6d24e1`. No incluye pruebas en producción ni verificación dinámica de los endpoints.